



Name: _____ Cohort/Batch: _____ Date: _____ Score: _____

JANUSGRAPH PRACTICE SHEET - 10 CONCEPTUAL Q&A

10 conceptual Q&A Databases

TOTAL: 10 QUESTIONS

Q1 What type of database is JanusGraph and what are its core strengths? Score Model

Q6 How do you monitor and tune slow queries in JanusGraph? Graph Ability

Q2 How does JanusGraph guarantee consistency and handle transactions? Architecture

Q7 What backup and restore strategies does JanusGraph support? Lifecycle

Q3 What index types does JanusGraph support and when do you use each? Indexing

Q8 How do you handle schema migrations in JanusGraph? Compliance

Q4 How do you design a schema or data model in JanusGraph? Graph Modeling

Q9 What security features does JanusGraph provide? Testing

Q5 How does JanusGraph scale horizontally? SSO / IdP

Q10 What are common anti-patterns when using JanusGraph in production? Monitoring



ANSWER KEY & EXPLANATORY SOLUTIONS

Comprehensive verified answers, best-practice configs, security controls & reference

VERIFIED SOLUTIONS

A1 JanusGraph is a relational, document, key-value, graph, or time-series store. Optimization

JanusGraph is a relational, document, key-value, graph, or time-series store. Its strengths such as ACID guarantees, horizontal scale, or flexible schema guide the workloads it is best suited for.

A6 JanusGraph provides a slow query log, explain/explain plan, or profiling tools to surface inefficient queries. Observability

JanusGraph provides a slow query log, explain/explain plan, or profiling tools to surface inefficient queries. Common fixes include adding indexes, rewriting queries, or increasing cache size.

A2 JanusGraph provides either full ACID transactions or eventual consistency depending on its CAP theorem positioning. Primitives

JanusGraph provides either full ACID transactions or eventual consistency depending on its CAP theorem positioning. Transaction support varies from none (simple K/V stores) to serializable isolation (relational DBs).

A7 JanusGraph supports logical backups (export SQL or BSON), physical backups (filesystem snapshot), and continuous replication to a standby. Automation

JanusGraph supports logical backups (export SQL or BSON), physical backups (filesystem snapshot), and continuous replication to a standby. Point-in-time recovery requires WAL/oplog archiving on top of backups.

A3 JanusGraph offers B-tree, hash, full-text, spatial, or composite indexes depending on the engine. Hardening

JanusGraph offers B-tree, hash, full-text, spatial, or composite indexes depending on the engine. Choose based on query patterns: B-tree for range queries, hash for equality lookups, full-text for search.

A8 Schema migrations in JanusGraph are managed with versioned migration files applied in order by a migration tool. Governance

Schema migrations in JanusGraph are managed with versioned migration files applied in order by a migration tool. Migrations should be idempotent and tested in staging before production to avoid downtime.

A4 Schema design in JanusGraph involves normalizing relations (relational DB) or denormalizing for access patterns (document/key-value). Primitives

Schema design in JanusGraph involves normalizing relations (relational DB) or denormalizing for access patterns (document/key-value). Understanding query needs upfront prevents costly migrations later.

A9 JanusGraph supports role-based access control, encrypted connections (TLS), encryption at rest, and audit logging. Encryption

JanusGraph supports role-based access control, encrypted connections (TLS), encryption at rest, and audit logging. Always restrict user privileges to the minimum needed and disable anonymous access in production.

A5 JanusGraph scales via replication (read replicas) for read-heavy workloads and sharding (partitioning data by key) for write-heavy ones. Federation

JanusGraph scales via replication (read replicas) for read-heavy workloads and sharding (partitioning data by key) for write-heavy ones. Some JanusGraph implementations handle this automatically; others require manual configuration.

A10 Common mistakes include selecting all columns instead of needed fields, missing indexes on frequently queried columns, running migrations without backups, and storing large blobs directly in the database instead of object storage. Optimization

Common mistakes include selecting all columns instead of needed fields, missing indexes on frequently queried columns, running migrations without backups, and storing large blobs directly in the database instead of object storage.